

LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2

Fondasi Arsitektur Internet of Things (IoT) dan Mikrokontroler



Disusun Oleh :

Muhamad Ramadhani (25782045)

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

1.4 Persiapan Praktikum

1.4.1 Kebutuhan Perangkat Keras

Pastikan komponen berikut tersedia dan siap digunakan:

1. *Board* NodeMCU ESP8266 (1 buah)
2. Kabel Data Micro USB (1 buah)
3. *Breadboard* / *Project Board* (1 buah)
4. LED 5mm (1 buah)
5. Resistor 220 Ohm (1 buah, sebagai pembatas arus LED)
6. Resistor 10k Ohm (1 buah, sebagai *pull-down* tombol)
7. *Tactile Push Button* / Sakelar Tekan (1 buah)
8. Kabel *Jumper* tipe *Male-to-Male* (secukupnya)

1.4.2 Konfigurasi Lingkungan Pengembangan (Arduino IDE)

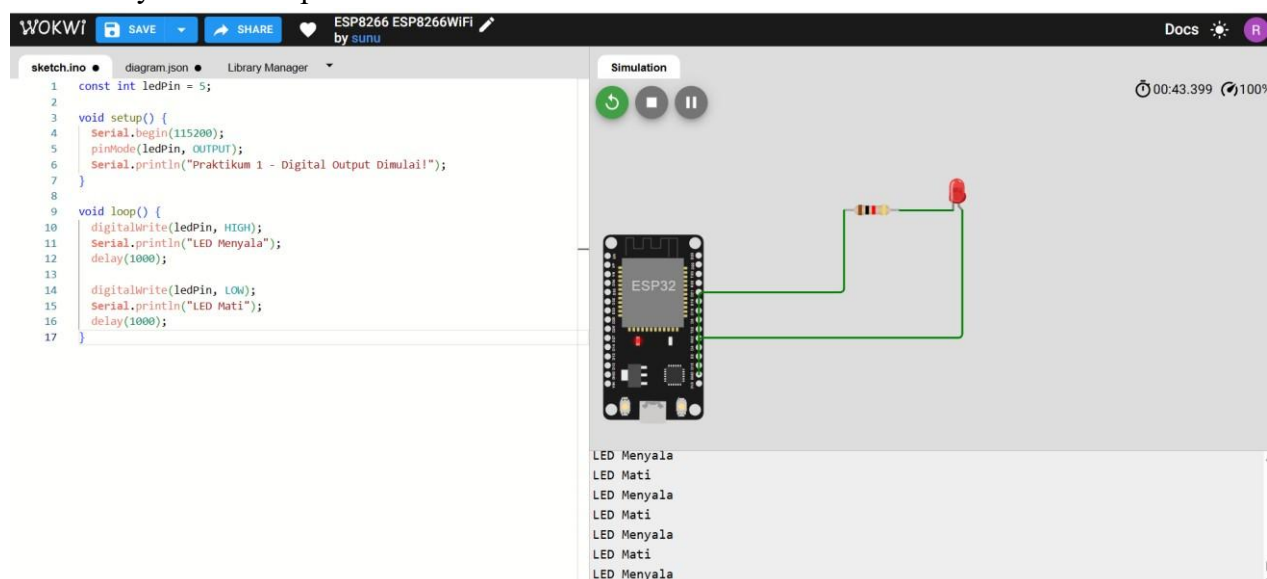
Sebelum memprogram, instal *board package* ESP8266 pada perangkat lunak Arduino IDE agar mikrokontroler dikenali.

1. Buka aplikasi Arduino IDE.
2. Buka menu File > Preferences.
3. Pada kolom Additional Boards Manager URLs, tempelkan tautan berikut:
http://arduino.esp8266.com/stable/package_esp8266com_index.json
4. Buka menu Tools > Board > Boards Manager...
5. Ketik "esp8266" di kolom pencarian, kemudian klik Install pada paket esp8266 by ESP8266 Community. Tunggu hingga proses unduhan selesai.
6. Konfigurasi parameter *board* Anda pada menu Tools:
 - o Board: Pilih NodeMCU 1.0 (ESP-12E Module).
 - o Port: Pilih *port COM* yang muncul saat NodeMCU dihubungkan ke laptop via USB.

1.5 Praktikum

1.5.1 Praktikum 1: *Digital Output* (Menyalakan LED)

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.



Langkah Kerja Rangkaian:

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - o Kaki yang **lebih panjang** adalah **Anoda (Positif)**.
 - o Kaki yang **lebih pendek** (dan terdapat sisi datar di tepi plastik kepala LED) adalah **Katoda (Negatif)**.
3. Hubungkan pin **D1 (GPIO 5)** dari NodeMCU ke kaki **Anoda** LED menggunakan kabel *jumper*.
4. Hubungkan kaki **Katoda** LED ke salah satu ujung **Resistor 220 Ohm**.
5. Hubungkan ujung **Resistor** lainnya ke jalur **GND** pada NodeMCU. (*Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar*).

Langkah Kerja Pemrograman:

1. Buka Arduino IDE, klik **File > New Sketch**.
2. Tuliskan kode program berikut:

```
const int ledPin = 5; void
setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Praktikum 1 - Digital Output Dimulai!"); }
```

```
void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED Menyala"); delay(1000);
```

```
  digitalWrite(ledPin, LOW);
  Serial.println("LED Mati"); delay(1000);
}
```

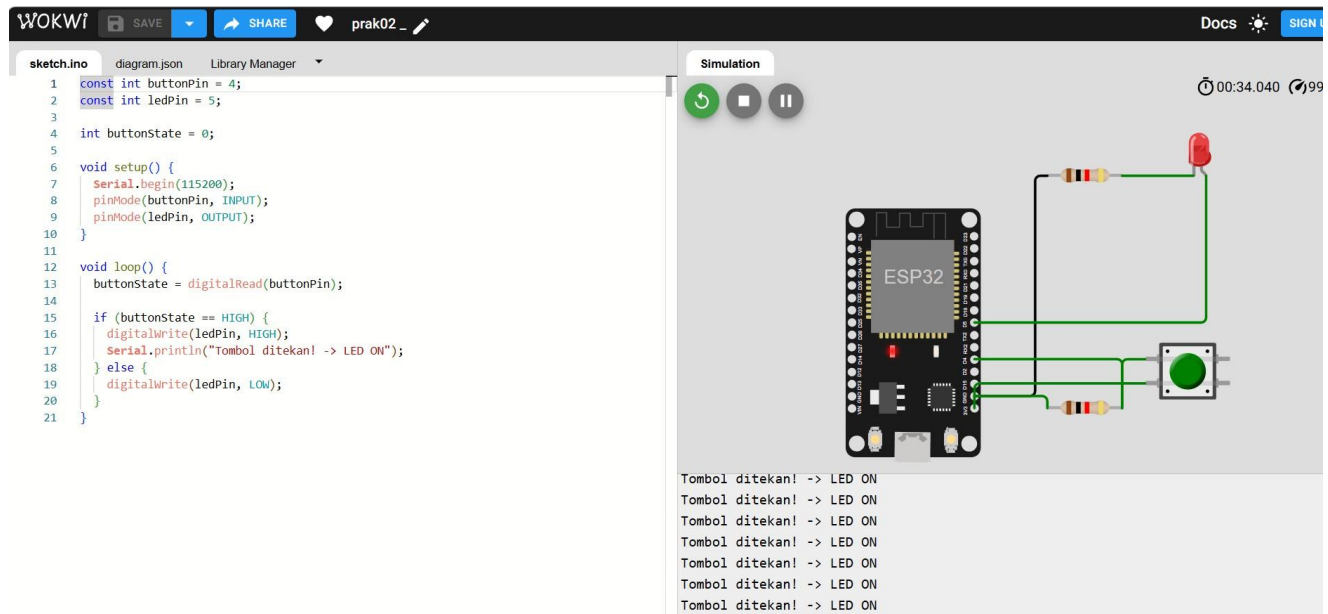
3. Klik tombol **Upload** (ikon panah ke kanan). Tunggu hingga muncul notifikasi *Done uploading*.
4. Buka **Serial Monitor** (ikon kaca pembesar di sudut kanan atas IDE).
5. Ubah pengaturan *baud rate* pada pojok kanan bawah Serial Monitor menjadi **115200 baud**.
6. Amati hasilnya: LED fisik berkedip setiap 1 detik selaras dengan teks status yang dicetak pada Serial Monitor.

Penjelasan Singkat Kode:

- `setup()`: Blok fungsi yang dieksekusi hanya **satu kali** saat NodeMCU dinyalakan atau di-*reset*. Biasanya digunakan untuk pengaturan awal.
- `loop()`: Blok fungsi yang dieksekusi secara **berulang-ulang** (terus menerus) selama NodeMCU memiliki daya.
- `pinMode(ledPin, OUTPUT)`: Memberi instruksi bahwa pin D1 (GPIO 5) akan digunakan sebagai jalur keluar (*Output*) tegangan.
- `digitalWrite(ledPin, HIGH)`: Memberikan tegangan 3.3V secara digital ke pin tersebut, sehingga arus mengalir dan LED menyala.
- `delay(1000)`: Menghentikan sementara eksekusi program (jeda) selama 1000 milidetik (1 detik).

1.5.2 Praktikum 2: *Digital Input* (Membaca *Push Button*)

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor *pull-down* eksternal.



Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin **D1** dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan **kaki pertama** *push button* ke pin **3V3** (3.3V) pada NodeMCU.
4. Hubungkan **kaki kedua** *push button* ke pin **D2 (GPIO 4)** NodeMCU.
5. Pada **kaki kedua** yang sama, pasang sebuah **Resistor 10k Ohm**, lalu hubungkan ujung lain dari resistor tersebut ke jalur **GND** NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program berikut:

```
const int buttonPin = 4;
```

```
const int ledPin = 5;
```

```
int buttonState = 0;
```

```
void setup() {
```

```

Serial.begin(115200);

pinMode(buttonPin, INPUT);

pinMode(ledPin, OUTPUT);

} void loop() {  buttonState =

digitalRead(buttonPin);  if

(buttonState == HIGH)

{  digitalWrite(ledPin, HIGH);

  Serial.println("Tombol ditekan! -> LED ON");

} else

{  digitalWrite(ledPin,

LOW);

}

}

```

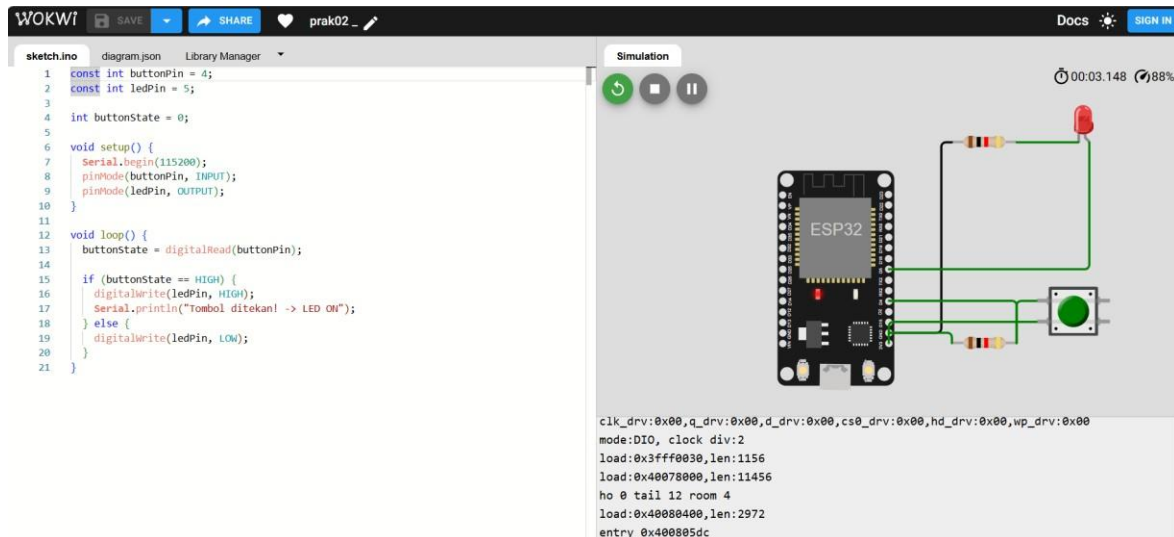
3. **Upload** program ke dalam mikrokontroler.
4. Buka **Serial Monitor**.
5. Uji fungsionalitasnya: Saat tombol ditekan dan ditahan, LED akan menyala terang. Saat tombol dilepaskan, LED langsung padam.

Penjelasan Singkat Kode:

- `pinMode(buttonPin, INPUT)`: Menetapkan pin D2 (GPIO 4) sebagai jalur masuk (*Input*) untuk mendeteksi / membaca tegangan dari luar.
- `digitalRead(buttonPin)`: Fungsi yang bertugas membaca status tegangan pada pin saat ini. Hasilnya akan berupa HIGH (ada tegangan) atau LOW (tidak ada tegangan).
- `if (buttonState == HIGH)`: Blok logika *conditional*. Karena kita menggunakan rangkaian resistor *pull-down*, menekan tombol akan mengalirkan arus 3.3V ke pin mikrokontroler. Sehingga, status terbaca HIGH dan kondisi if terpenuhi (LED dinyalakan).

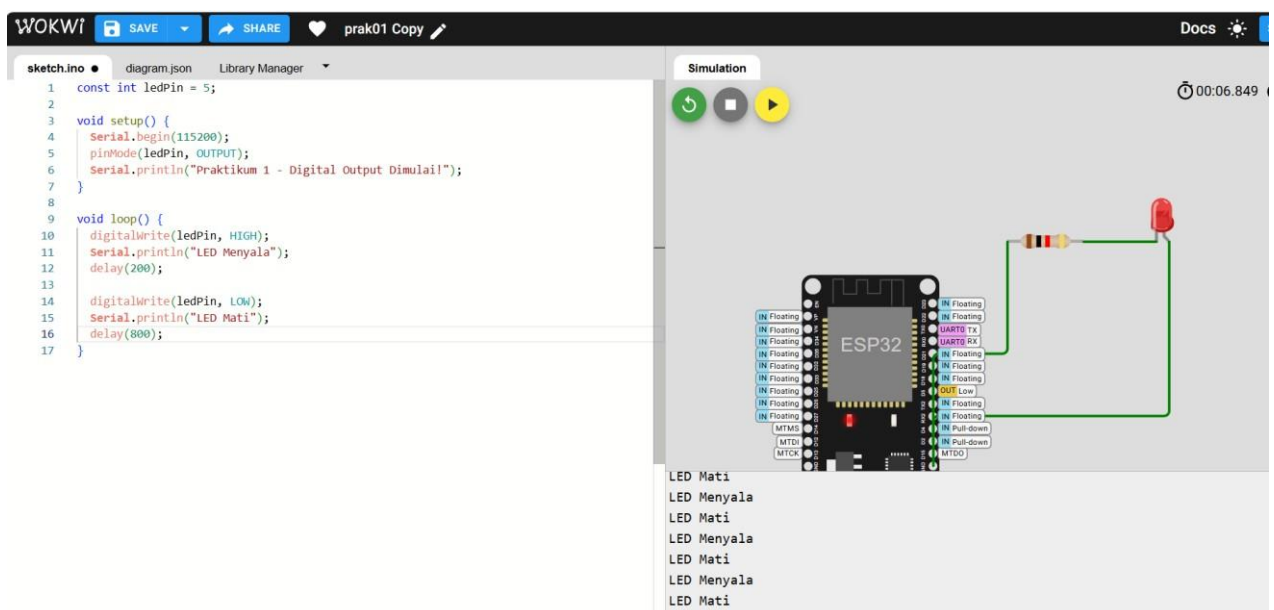
1.6 Latihan

- Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?



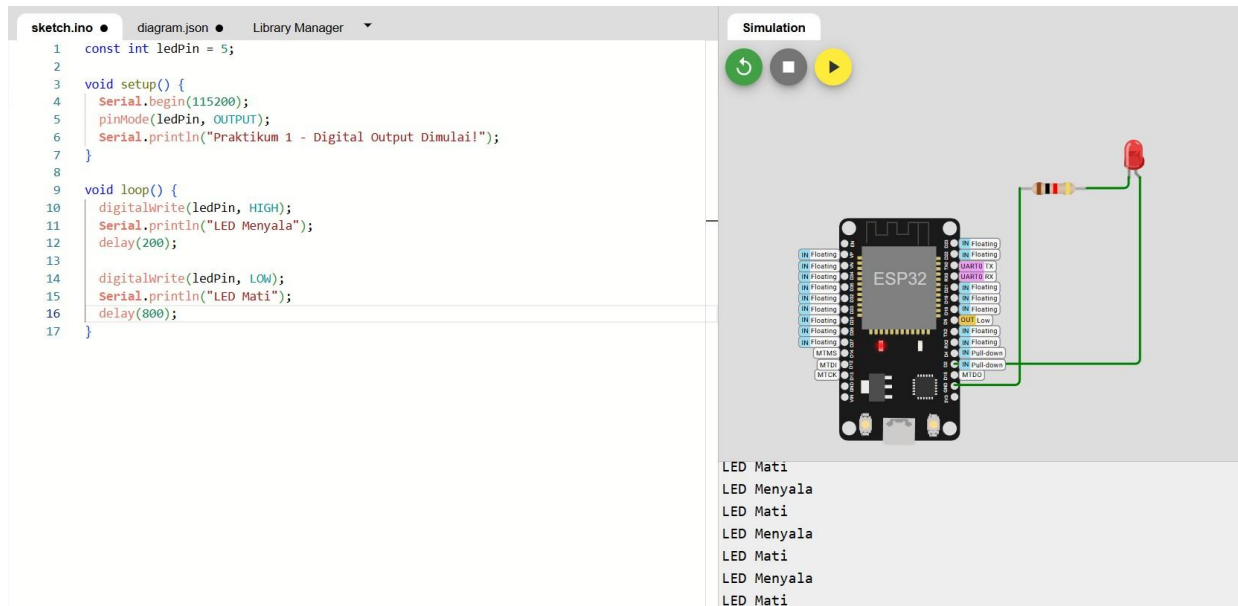
Percobaan pada Wokwi menunjukkan bahwa rangkaian Active-High membuat D8/GPIO15 menjadi HIGH ketika tombol ditekan. Jika kondisi tersebut terjadi tepat saat NodeMCU ESP8266 diberi daya, GPIO15 yang seharusnya LOW untuk boot normal dapat menyebabkan NodeMCU gagal melakukan boot normal atau masuk ke mode boot yang berbeda. Oleh karena itu, D8/GPIO15 tidak cocok digunakan sebagai input Active-High yang dapat berubah menjadi HIGH pada saat startup.

- Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.



Dua fungsi delay() yang diubah adalah delay(1000) menjadi delay(200) setelah LED menyala, dan delay(1000) menjadi delay(800) setelah LED mati. Dengan demikian, LED menyala selama 200 ms dan mati selama 800 ms sehingga kedipan LED menjadi tidak simetris.

- **Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

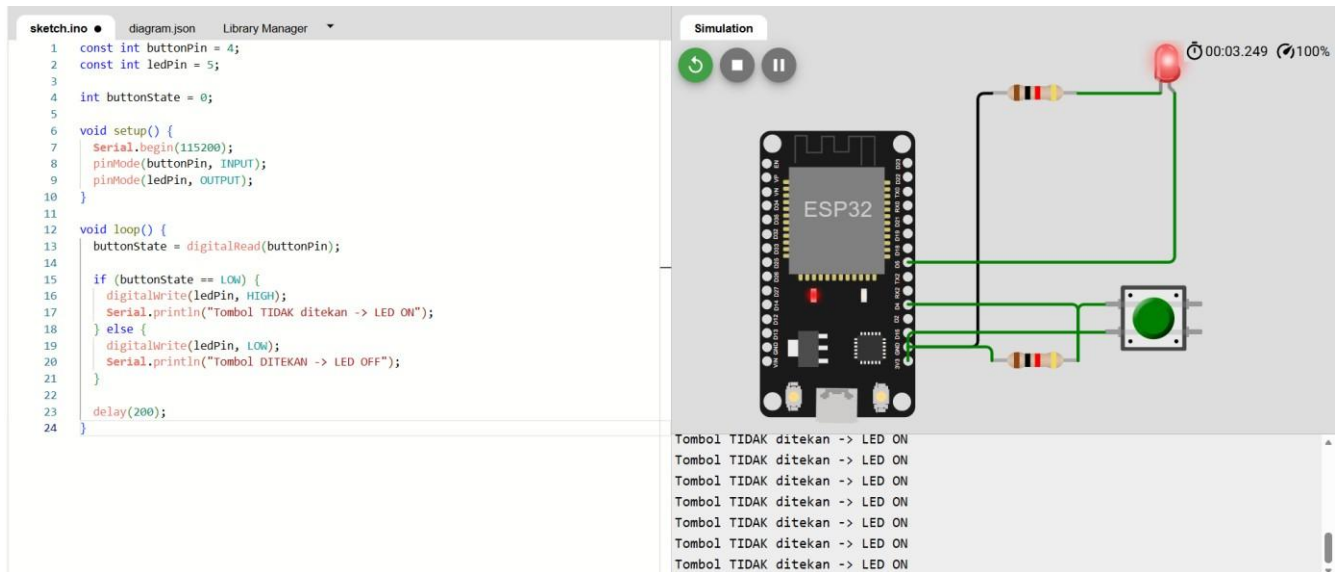


Jika kaki LED dipasang terbalik, yaitu **Anoda dihubungkan ke GND dan Katoda ke pin D1**, maka LED tidak akan menyala. Hal ini karena LED merupakan komponen yang memiliki arah aliran arus. Pada kondisi tersebut, arah pemasangan LED berlawanan dengan arah aliran arus yang seharusnya, sehingga arus tidak dapat mengalir melalui LED dengan benar.

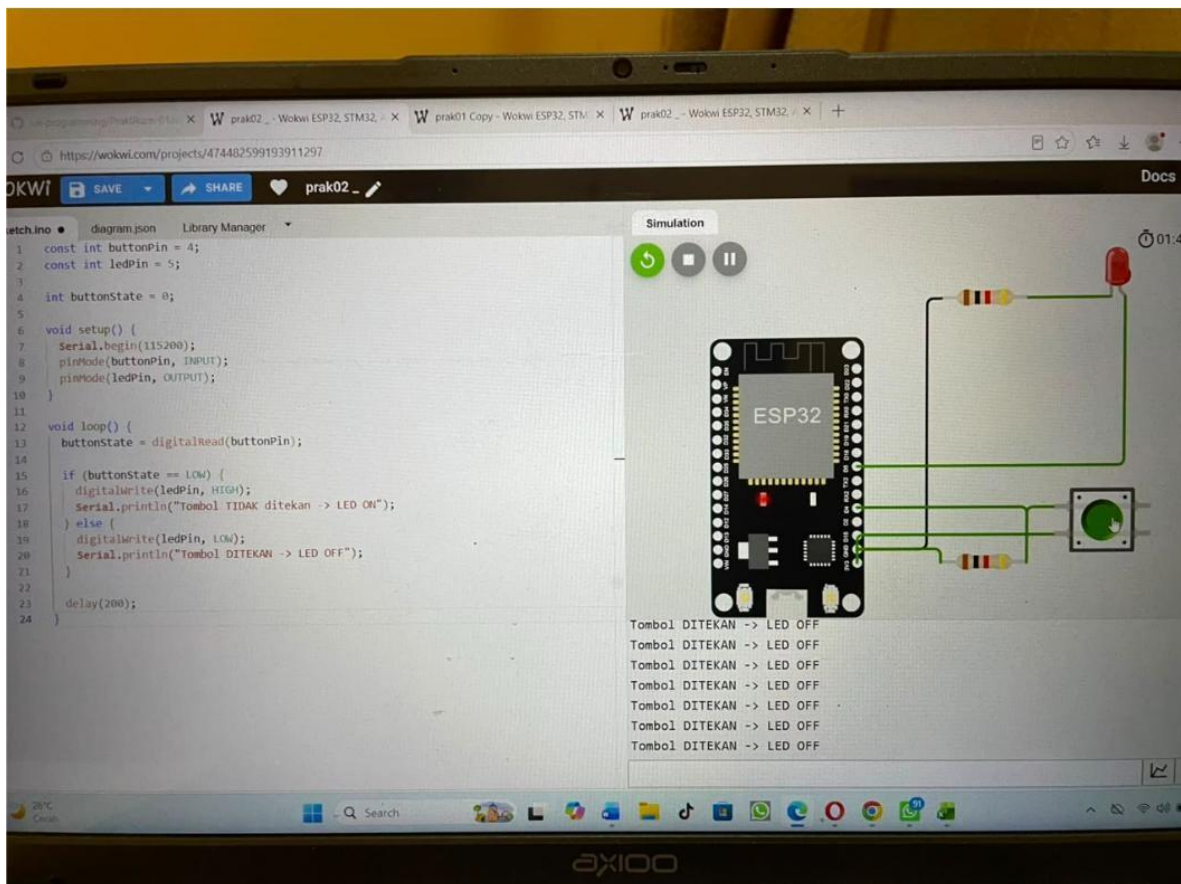
Program **tidak mengalami error saat kompilasi**, karena posisi atau arah pemasangan LED merupakan masalah pada **rangkaian hardware**, bukan kesalahan pada kode program. Program tetap dapat dijalankan dan pin D1 tetap memberikan kondisi HIGH dan LOW sesuai perintah. Program tetap berjalan dan tidak error, tetapi LED tidak menyala karena LED dipasang terbalik.

Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

SCREENSHOT POSISI TOMBOL TIDAK DI TEKAN



SCREENSHOT POSISI TOMBOL DI PENCET



Kondisi if-else diubah dengan menjadikan LOW sebagai kondisi LED menyala. Jadi, ketika tombol tidak ditekan, input bernilai LOW sehingga LED menyala. Sebaliknya, ketika tombol ditekan, input menjadi HIGH sehingga LED dimatikan.

1.7 Tugas

Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

Modifikasi Sistem Sakelar *Toggle (Latching)*

- **Tujuan:** Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*. • **Tingkat Kesulitan:** Menengah
- **Instruksi:**
 - i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
 - ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali *lalu dilepas*, LED **menyala dan tetap menyala**.
 - Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED **mati dan tetap mati**.
 - iii. *Petunjuk:* Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).
- **Kriteria Keberhasilan:**
 - Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
 - Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
 - Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.

HASIL DARI TUGAS PRAKTIKUM

```
1 const int buttonPin = 4;
2 const int ledPin = 5;
3
4 // Menyimpan status LED: false = mati, true = menyala
5 bool ledState = false;
6
7 // Menyimpan kondisi tombol sebelumnya
8 int lastButtonState = LOW;
9
10 void setup() {
11   Serial.begin(115200);
12
13   pinMode(buttonPin, INPUT);
14   pinMode(ledPin, OUTPUT);
15   digitalWrite(ledPin, LOW);
16 }
17
18
19 void loop() {
20   int buttonState = digitalRead(buttonPin);
21
22   if (buttonState == HIGH && lastButtonState == LOW) {
23     ledState = !ledState;
24     digitalWrite(ledPin, ledState);
25
26     if (ledState) {
27       Serial.println("LED ON");
28     } else {
29       Serial.println("LED OFF");
30     }
31
32     delay(200);
33   }
34   lastButtonState = buttonState;
35 }
```

Simulation

load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
LED ON
LED OFF
LED ON

Pada tugas ini tombol diubah menjadi sakelar toggle dengan menggunakan variabel boolean bernama ledState untuk menyimpan kondisi LED. Setiap kali tombol mengalami perubahan dari LOW ke HIGH, nilai ledState dibalik menggunakan !ledState. Jika sebelumnya LED mati, maka LED menjadi menyala, dan sebaliknya. Variabel lastButtonState digunakan untuk mendeteksi satu kali tekanan tombol sehingga LED tidak berubah terus menerus ketika tombol ditahan. delay(200) digunakan sebagai debounce untuk mencegah pantulan mekanis tombol menyebabkan satu tekanan terbaca beberapa kali.

BERIKUT PENJELASAN KODE YANG SAYA MODIFIKASI AGAR SESUAI DENGAN INTRUKSI TUGAS

```
const int buttonPin = 4; const int ledPin = 5;
```

```
// Menyimpan status LED: false = mati, true = menyala bool  
ledState = false;
```

```
// Menyimpan kondisi tombol sebelumnya int  
lastButtonState = LOW;
```

```
void setup() {  
  Serial.begin(115200);  
  
  pinMode(buttonPin, INPUT);  
  pinMode(ledPin, OUTPUT);  
  // Saat pertama dinyalakan, LED dalam keadaan mati  
  digitalWrite(ledPin, LOW);  
}
```

```
void loop() {  int buttonState =  
digitalRead(buttonPin);  
  // Mendeteksi tombol baru saja ditekan  
  // LOW -> HIGH berarti terjadi satu kali tekanan  if (buttonState  
== HIGH && lastButtonState == LOW) { // Membalik status  
LED: OFF menjadi ON atau ON menjadi OFF    ledState  
= !ledState;
```

```
  digitalWrite(ledPin, ledState);
```

```
  if (ledState) {  
    Serial.println("LED ON");  
  } else {  
    Serial.println("LED OFF");  
  }  
  // Debounce untuk mencegah satu tekanan terbaca berkali-kali  
  delay(200);  
}
```

```
// Menyimpan kondisi tombol untuk pengecekan berikutnya  
lastButtonState = buttonState;  
}
```